OPERATING SYSTEMS – ASSIGNMENT 1

TASK 10 – Measure and Compare (4 Marks)
CO1–CO5 | Bloom's Level: K5 – Evaluate

Introduction

Measurements help check whether the actual operating system behaves as expected. The hospital should test system-call counts, execution time and process or thread creation time on the Linux system used for development.

1. Measure system calls for different buffer sizes

Use the completed file-copy program from Task 6. Run it with different buffer sizes and record the program's read() count. Then use **strace -c ./copy** to view the system-call summary. The assignment's expected read() counts for the 10,000-byte file are:
1 byte buffer – 10,001 read() calls
512 byte buffer – 21 read() calls
4096 byte buffer – 4 read() calls
The strace total may be higher because program startup and library functions can also make read() calls. Identify the copy operation when comparing results.

2. Measure user-mode and kernel-mode time

Use the Linux command **/usr/bin/time -v ./copy** or **time ./copy**. Record the elapsed (real) time, user CPU time and system CPU time. User time measures CPU work in user mode. System time measures CPU work in the kernel on behalf of the program. Repeat each test several times and compare the results rather than relying on one run.

3. Measure creation of 1,000 processes and threads

For process creation, write a C program that calls fork() 1,000 times and waits for every child using waitpid(). Record the elapsed time. For thread creation, use pthread_create() to create 1,000 threads and pthread_join() to wait for them. Compile the thread program with **gcc -std=c11 -Wall -Wextra -pthread threads.c -o threads**. Measure both programs using **/usr/bin/time**. Use the same computer and avoid running heavy background applications during the test.

4. Results table to fill with your own measurements

The assignment requires your own measured evidence. Do not copy invented timing values. Enter the results obtained on your Linux computer in this table.

Measurement table

Test | Your measured result
read() calls, 1-byte buffer | _____
read() calls, 512-byte buffer | _____
read() calls, 4096-byte buffer | _____
Copy program real time | _____
Copy program user time | _____
Copy program system time | _____
1,000 fork() processes real time | _____
1,000 pthreads real time | _____

5. How measurements support the earlier judgements

The expected read() counts show that increasing the buffer size can greatly reduce the number of system calls. The timing measurements indicate whether fewer system calls also reduce execution time on the test machine. Process and thread measurements show the cost of creating separate processes compared with threads. Results may vary with CPU speed, memory, Linux version, system load and compiler settings.

6. Alternative concept: threads instead of processes

Threads generally require less memory and may be faster to create because they share the same process address space and many resources. Separate processes have their own virtual address spaces and provide stronger isolation. Threads need less duplicated process state but require careful synchronisation when sharing data. For independent commands or tasks that must be isolated, processes are safer. For closely related tasks that share data, threads may be more efficient. The actual difference should be confirmed by the measurements.

Conclusion

The hospital should use measured results to choose buffer sizes and process-management methods. The final report must include the actual output and timings collected from the student’s Linux system.

Prepared from the corresponding task in the uploaded CSA04 Operating Systems Assignment 1 document. Add your name, register number, required screenshots and your own measured evidence wherever requested before submission.